Project Audit — Indigenomics RAP Data Portal

Prepared for the CS7980 showcase and client handoff · 2026-07-30 (updated 2026-08-02: deploy runbook now shipped, WAF added, review-queue edit/verify + quote-locate)

This document is a top-down audit of the portal: what it is, what it costs (with **real AWS billing data**), what would need to change to run it in the client's own AWS account, and what the client needs for a clean transition.

---

1. Executive summary

The portal is a **Next.js 14 (App Router) application deployed via SST v4 + OpenNext on AWS** — a server-rendered app running on Lambda behind CloudFront, not a static S3 site. It does two things:

1. **RAP (Reconciliation Action Plan) extraction and analysis** — upload a PDF, OCR it, extract *grounded* commitments with an LLM (every value carries a verbatim source quote + page), human-review, and surface them as a searchable "RAP Index."
2. **A legal-cases lens** — a hybrid-search corpus of ~43k Canadian legal cases with grounded briefs and single-case Q&A.

It serves three personas: **company**, **Indigenous supplier**, and **Indigenomics institute**.

Architecture is strong. Every domain sits behind a clean seam (in-memory mock ⇄ DynamoDB, and mock ⇄ real-AI), so the UI never touches infrastructure directly and everything is testable offline.

Cost reality: gross AWS usage for 2026 was **$371.49**, **~98% of it Bedrock LLM inference**, and **almost entirely offset by AWS credits** (net out-of-pocket ≈ $0.06). Non-AI infrastructure ran **~$1/month**. The catch: those credits won't exist in the client's own account.

Handoff is bounded — a short list of hard blockers (two hardcoded sandbox ARNs, one out-of-band data table, re-provisioned secrets/SES/OIDC, and a region-strategy decision), detailed in §4.

Main caveats for an auditor: demo-mode is the default everywhere (the app runs a mock data layer and a shared demo password by default — note the dashboard *dataset* itself is real: 100 genuine organizations sourced from public disclosures, with only 3 fictional demo accounts), a legacy "report→confirm" data layer coexists with the newer RAP-extraction layer, and the app runs in a **governance-constrained sandbox account** where Textract and CloudTrail are blocked by parent-org SCPs.

---

2. Top-down architecture

2.1 Deploy topology

`sst.aws.Nextjs("Web")` provisions, per stage: a **CloudFront distribution → S3 static assets + a server Lambda (2048 MB) + an image-optimizer Lambda + a revalidation SQS**

queue + an ISR DynamoDB table. There is **no custom domain** — every deploy gets a random `*.cloudfront.net` URL. Two live deployments exist: current **ca-central-1** (data-residency migration, real data) and a **us-east-1** backup. Legal cases are always served cross-region from us-east-1.

Stage model (this shapes behavior everywhere):

Stage	Region	Extraction engine	Doc loader	Observability
`production`	us-east-1	BDA (Bedrock Data Automation)	Textract	**Yes** (DLQ, X-Ray, 6 alarms, dashboard)
`ca`	ca-central-1	Bedrock (Textract→Claude)	text-layer*	**Yes** (DLQ, X-Ray, 6 alarms, dashboard)
dev / other	us-east-1	mock	textract	No

The observability stack is gated to the two stages that run **real extraction** (`observe = isCa || isProd` in `sst.config.ts`); each gets its own stage-scoped copy. Mock/dev stages skip it.

The same `observe` gate now also attaches an **AWS WAFv2 WebACL to the CloudFront distribution** (rate-limit 1,000/5-min/IP + AWS managed CommonRuleSet + KnownBadInputs), created in us-east-1 and shipped **count-first** — see §5, item 5.

* The `ca` stage ships on a **PDF embedded-text-layer** loader because **Textract is SCP-blocked** in this org — see §4.3.

2.2 Compute

- **Web server Lambda** — 2048 MB; carries Bedrock/Textract/SES/Lambda-invoke IAM.
- `RapExtract` — 1536 MB, 900 s timeout; async fire-and-forget extraction worker (X-Ray Active on `ca` + `production`).
- `BriefGen` — 2048 MB, 300 s; async legal-brief + case-Q&A worker.
- **Stream processors** — `RollupAggregator` (RapData stream → recompute commitment rollups), `AlignmentEngine` (Commitments stream → recompute supplier matches).
- **Crons** — `StuckJobMonitor` (15 min, `ca` + `production`), `CaseMonitor` (weekly new-case scan), `NotifyDigest` (weekly overdue-milestone email, prod-only).

All functions are **x86_64** (arm64 is never set — a free ~20% compute saving left on the table).

2.3 Data layer — DynamoDB single-table designs

Each domain owns its own table sharing a generic `PK/SK + GSI1 + GSI2` shape (all on-demand billing): `DataPortal`, `RapData`, `Commitments`, `Alignment`, `Notifications`, `RapSurvey`, plus the out-of-band `LegalCases` (~43k items, wired by ARN, pinned to us-east-1). `RapData` and `Commitments` have DynamoDB Streams; `RapData` also has point-in-time recovery. The anti-hallucination core is `Grounded<T>` — an extracted value is only trusted if the model returns a verbatim source quote + page number.

2.4 Application domains (`src/lib/`)

Contract-first modules, each a `types.ts` interface + a mock/dynamo swap:

- **auth** — HMAC-signed session cookies; `getSession()` is the real security boundary.
- **repo** — legacy report→confirm portal (still backs `/report`, `/confirm`, `/analytics`).
- **rap** — the extraction pipeline: doc-loader → grounded LLM extraction → deterministic validation gates → human review → canonical RAP graph. The review queue supports **per-field edit + check-off** (reviewer corrects/verifies each flagged field before publish), and a **quote-locate** step recovers verbatim source citations + page for the BDA path (which grounds by confidence, not a text quote), so evidence cards and jump-to-PDF work on `production` too.
- **commitments** — the RAP Index.
- **alignment** — BM25 + embeddings supplier matching (never scores on the stub embedder).
- **cases** — the largest module: ingest/harvest (robots-compliant), hybrid BM25 + dense search with RRF, briefs, case-QA, monitoring.
- **governance** — OCAP data-residency classification (`public` vs `org_submitted`, fails closed).
- **identity / index-evidence** — Business-Number crosswalk + evidence-precedence tiers.

2.5 Personas and access control

Enforcement is **two-layer**: `middleware.ts` does fast, UX-only routing off the (unverified) cookie `kind`; **real authz is `getSession()`** on each page/action, which recomputes the HMAC and checks expiry. Company/supplier/indigenomics each get their own route set; legal cases are open-read to all. (Worth noting in an audit: the middleware guard itself does not verify the HMAC — by design; the server does.)

2.6 Maturity flags (real vs. mock)

- **Mock is the default** — real DB and real AI only engage when env flags are set (SST sets them on deployed stages).
- **Demo auth is intentionally insecure** — 103 demo *company* logins (plus the 10 supplier `@demo` accounts and `institute@demo`) share the password `demo-portal-2026`; must be purged before any real client production.
- **Legacy dual-model** — `src/lib/repo` (report→confirm) coexists with the RAP layer.
- **Known TODOs (none are blockers).** (a) The extraction pipeline's *optional* second-pass LLM-as-judge ("does each quote support its value?") is not implemented — safe to omit, because the existing validation gate + human review already decide what publishes, and the live engine study (`rap-engine-comparison.md`) found an LLM judge does not transfer to this data (κ=0).
  (b) `sectorFields`/`pillars` are **derived deterministically from the commitments**, not asked of the LLM — this is intentional (more reliable than a model guess), not an unfinished feature.
  (c) The `RAP_INDEX_SOURCE` cutover is partial: the Explore view already unions seeded + published-RAP data (`"merge"`), but the Table view still reads the seeded domain directly, so a full "published-RAP-only" cutover would also move the Table view onto the same seam.
- **Testing** — no unit framework, but ~90 `tsx` assertion/parity harnesses give strong scenario coverage of pure logic; no CI-enforced coverage thresholds.

---

3. Cost report (real billing data)

Source: AWS Cost Explorer, account `106189426706` (`isb` profile), Jan–Aug 2026, unblended cost. Pulled 2026-07-30.

3.1 The headline

Metric	Amount
Gross AWS usage, Jan–Aug 2026	**$371.49**
AWS credits applied	−**$371.44**
Tax	$0.02
Net out-of-pocket	≈ **$0.06**

The team has effectively paid nothing — the entire bill is covered by AWS credits held by the account/org. **This is the most important thing for the client to understand:** in their own account, with no credits, that ~$371 becomes real money.

3.2 Where the money goes — ~98% is LLM inference

Gross usage by service (Jan–Aug 2026):

Service	Gross $	Notes
Claude Haiku 4.5 (Bedrock)	$215.65	The volume workhorse (labeling, eval, extraction iterations)
Amazon Bedrock (generic)	$86.61	Llama 3.3 70B + Titan embeddings + BDA bill here
Claude Sonnet 4.5 (Bedrock)	$52.80	Extraction / briefs
Claude Sonnet 4.6 (Bedrock)	$8.09	Extraction (current default model)
Palmyra X5 (Bedrock)	$0.84	Experimental
Amazon Textract	$0.63	Low — consistent with the SCP block
— Bedrock LLM subtotal —	**~$364**	**~98% of gross**
EC2 / VPC (one-off compute)	~$4.37	Likely a corpus/embedding build box
DynamoDB	$1.46	
AWS Config	$0.53	Control Tower baseline
S3	$0.26	~1.1 GB search-index artifacts + uploads
CloudWatch / CloudFront / SQS / other	<$0.25	Within free tiers

Non-AI infrastructure — Lambda, CloudFront, DynamoDB, S3, SQS, CloudWatch — totaled ~$7 over seven months (~$1/month). The serverless stack fits inside AWS's always-free allowances at this traffic level.

3.3 Cost is bursty and development-driven

Gross usage by month:

Month	Gross $
Jan 2026	$4.06
Feb 2026	$0.94
Mar–Jun 2026	~$0.14 total
Jul 2026	**$366.35**

$366 of the $371 was July — the capstone crunch: re-running extraction over corpora, eval loops, re-labeling, brief generation. This is *development* volume, not steady-state serving. It tells the client that cost scales with **how much extraction/generation they run**, not with idle uptime.

3.4 Post-capstone maintenance cost (real-data-anchored)

1. **Keep it running, idle (no dev, occasional demo): ~$1–3/month** — the measured non-AI infrastructure rate, dominated by DynamoDB + S3 storage and the ca CloudWatch dashboard/alarms. No traffic → Lambda/CloudFront/Bedrock ≈ $0.
2. **Cold-store data only** (`sst remove` compute, keep S3 + a Dynamo export): **~$1/month**.
3. **Tear down entirely** (`sst remove --stage <stage>`): **$0**. Note `production` is `retain`, so its data buckets survive a remove and need manual cleanup.

The variable cost is entirely LLM inference, proportional to extraction/brief volume. As a planning anchor from real usage: heavy development ran ~$50–350/month; steady production serving to end users (no bulk re-extraction) would be a fraction of that.

3.5 Cheap wins if it stays running

Switch Lambdas to **arm64** (~20% compute), set **S3 lifecycle** on uploads/analytics, add **DynamoDB TTL** on extraction-job staging rows, drop the 30-day default **log retention** to 7–14 days, and route bulk labeling/eval to **Haiku** (already the pattern) rather than Sonnet where quality allows.

---

4. Sandbox → client-owned environment: what changes

Good news first: naming is SST-generated per stage, there is no custom domain to migrate, and there are **no non-AWS paid dependencies** — all LLM inference is Bedrock (no OpenAI/Anthropic API keys, no registrar). The migration is a bounded checklist.

4.1 Hard blockers

1. **Two hardcoded sandbox ARNs** in `sst.config.ts` (BDA project `…:106189426706:data-automation-project/c8c9dfbd3f8e` and profile) — only resolve in account `106189426706`, and only on the `production` stage. The client must **create their own BDA blueprint + project** (from `src/lib/rap/bda-blueprint.json`) and override `BDA_PROJECT_ARN` / `BDA_PROFILE_ARN`.
2. **`LegalCases` is the biggest un-templated dependency** — not SST-managed, created by an out-of-band `cases:*:cloud` pipeline, region-pinned to us-east-1 by ARN in three places, ~43k items. The client must either **receive an export** or **rebuild via the harvest pipeline** (which scrapes external court sites).
3. **Secrets / identity to re-provision:** the one `AuthSecret` SST secret (per stage, or deploy fails), a **GitHub OIDC provider + deploy role** (`AWS_DEPLOY_ROLE_ARN`), **SES verified sender/recipient** identities, and **Bedrock model access enabled** in their region.

4.2 Region and residency decision (the strategic one)

The current split is deliberate: platform data *can* rest in **ca-central-1**, but **BDA, Textract, Titan embeddings, and the Claude/Llama inference profiles are all pinned to us-east-1** because Canada is not a Bedrock inference geography (data at rest in Canada, inference geo-routes via the `us.` profile). The client must pick a strategy:

- **No residency requirement** → collapse everything to one region; much simpler.
- **Canadian residency required** → they inherit the same us-east-1 split for inference, or run the text-layer engine.

4.3 The SCP constraints *disappear* in their account

Because the sandbox sits under a Control Tower org owned by an external party (derekja@uvic.ca), **Textract and CloudTrail are SCP-blocked**, which is *why* the `ca` stage ships on the text-layer workaround and *why* X-Ray was chosen over CloudTrail. In a **client-owned account with no such SCPs**, those become capabilities they *gain* — they can flip back to `EXTRACTION_IMPL=bedrock` + `DOC_LOADER=textract` for full grounded OCR and enable CloudTrail. The workaround code stays but becomes optional. **Document that these limits are an artifact of this org, not inherent to the product**, so the client doesn't inherit a phantom limitation. The live **three-engine comparison (`./03-rap-engine-comparison.md`, n=8)** backs this: on an unrestricted account, `bedrock` + `textract` is the recommended primary (best coverage + real page numbers), with `textlayer` as the cheapest/best-grounding fallback and BDA only where speed beats provenance.

4.4 Data hygiene before handoff

- **Demo logins.** Seeding is scripted (`scripts/seed-org-logins.ts`, `scripts/seed-sst.ts`), and purging is scripted too (`scripts/purge-demo-logins.ts` — dry-run by default, `--apply` to delete). Before real use, delete the **103 `@demo` company accounts** (plus the 10 supplier `@demo` accounts) and retire the shared `demo-portal-2026` password. **Keep `institute@demo`** — the Indigenomics Institute staff account (a separate singleton, not one of the 103) — so the Institute retains access from day one, but **set it a real, private password** in place of the shared demo one. Optionally leave one or two demo company logins for the client to explore with. Use **`scripts/purge-demo-logins.ts`** for this — it deletes the `@demo` login accounts

except a keep-list (default: `institute@demo`), is **dry-run by default**, and deletes only with `--apply`.
- Review `org-bn-map.ts` — hand-curated Canadian Business-Number crosswalk, self-flagged "VERIFY BEFORE PROD MIGRATION."
- Replace **fixture corpora** (real Bank of Canada / RBC RAP content sits in `scripts/fixtures/` — copyright-sensitive) with the client's own documents.

---

5. Recommendations for a smooth delivery and transition

1. **A one-page runbook / deploy guide. ✓ DONE — `DEPLOY-RUNBOOK.md`.** A single linear "zero → running" checklist now exists (prerequisites, per-account setup, the env-var table, region/residency strategy, BDA setup, seed + data hygiene, real-AI turn-on, verify, cost/teardown, troubleshooting). It supersedes the scattered `deploy.md` / `deploy-rap.md` for onboarding a new account.
2. **An architecture + data-flow diagram set.** The 9-diagram gallery is the backbone; add one **"client deployment topology"** diagram showing their-account resources.
3. **A data handoff decision + package.** Decide now whether `LegalCases` and the platform tables ship as **exports** or **rebuild scripts**, and document the choice. This is the single most likely thing to trip up a transition.
4. **A version-drift cleanup pass.** The repo says "SST v3" in comments/prose but pins **v4.15.2**; several config shapes carry "verify against the installed version" warnings. Reconcile this before the client's first `sst deploy`.
5. **A "sandbox → production hardening" checklist. AWS WAF is now in place ✓** — a WAFv2 WebACL (rate-limit 1,000/5-min/IP + AWS managed CommonRuleSet + KnownBadInputs) is attached to the CloudFront distribution on `ca` + `production`, created in us-east-1 and shipped **count-first** (rules count, nothing is blocked) so false positives surface before enforcement. **Remaining hardening:** flip the WAF from count to block (`WAF_BLOCKING=true`) after observing count-mode metrics; custom domain + ACM cert; real secrets; purged demo accounts; S3 lifecycle + log retention; arm64. Frame as "what to do before real users."
6. **A cost & teardown note.** The scenarios in §3.4, the `sst remove` teardown steps, the reminder that `production` is `retain`, and — most importantly — **that today's bill is credit-covered and becomes real spend (mostly Bedrock) in their account**. Recommend they set an AWS Budget alarm on day one.
7. **Clear ownership of the external dependency.** The Textract/CloudTrail SCPs belong to *this* org. Make explicit that these constraints do not travel with the product.

Appendix — top risks / flags

- Account ID `106189426706` baked into BDA ARNs (prod stage).
- `LegalCases` out-of-band, region-pinned, large — the un-templated dependency.
- Demo credentials (`demo-portal-2026`, 103 `@demo` accounts) shipped by default.
- **Cost is credit-subsidized** — net ≈ $0.06 hides ~$371 of gross Bedrock usage.
- SST v3-vs-v4 documentation drift.

- OpenNext implicit resources (revalidation queue, ISR table, image Lambda) not named in config — enumerate from deployed state during migration.
- Dense retrieval is **on by default** (`EMBED_PROVIDER=bedrock`): the `vectors.bin` artifact was quantized (~985 MB → ~31 MB resident), resolving the earlier OOM at 3008 MB that had made it opt-in.
- Legacy report→confirm layer coexisting with the RAP layer (dual data model).

Cost figures are real AWS Cost Explorer data (unblended, account `106189426706`, Jan–Aug 2026). Architecture and coupling findings are from a source audit of the `portal` repo at the date above.